

PLAYHACK EXPLAINED

A campus sports facility booking system in which the winner of a race for one court is decided by Postgres, not by application code — and the complete engineering account of why that choice was made, how it was built, how it is deployed, and how every claim in it was verified.

ONE

Overview

The problem. IIT Guwahati has twelve bookable sports facilities and roughly six thousand students. At 6:00 PM, many of them want the same court. A booking system must confirm exactly one of those requests and reject the rest, without ever corrupting the record of who holds what.

What almost everybody builds. Check whether the slot is free, then write the booking if it looked free. Both statements are individually reasonable. Between them is a gap, and under real load another request commits inside it. Two students walk to the same court, and nothing in the logs looks like an error.

What this does instead. There is no check. A booking is a row carrying a time range, and one Postgres exclusion constraint makes it physically impossible to store two overlapping ranges for the same facility. The write *is* the decision. The losers are rejected by the database with SQLSTATE **23P01** before they can become rows.

The measured result

Fifty simultaneous requests for one court, fired at the deployed application. Through a textbook check-then-write: **50 confirmed** and **1,225 overlapping pairs** in the table. Through the constrained path, identical traffic: **1 confirmed, 49 rejected, 0 overlapping pairs** across every confirmed row in the database.

What is in this document

01	Overview
02	The project, explained for anyone
03	The concurrency problem, in depth
04	The mechanism: exclusion constraints over ranges
05	Data model
06	The write path, statement by statement
07	Idempotency
08	Losing well: typed rejections and alternatives
09	Cancellation and waitlist promotion
10	Maintenance closures
11	The fair draw
12	Availability is derived, never stored
13	Real time, and why the payload is a hint

14	The interface
15	Deployment and topology
16	Performance engineering
17	Verification strategy
18	Security posture and threat model
19	Limitations, and what we would do next
A	Appendix: API reference
B	Appendix: measured data
C	Appendix: repository map and glossary

TWO

The project, explained for anyone

Imagine a single badminton court and a noticeboard beside it where students write their name against an hour. At 6:00 PM fifty students arrive at the noticeboard at once, all holding pens, all looking at the same empty line for 7:00 PM.

A careless system is a noticeboard with fifty pens. Everybody looks, sees the line is empty, and writes. Fifty names now occupy one line. Nobody did anything wrong; the noticeboard simply had no way to stop the second pen.

The obvious fix is a supervisor who looks at the board before letting you write. That is what most booking software does — a check, then a write. It fails for a subtle reason: between the moment the supervisor looks and the moment your pen touches the paper, somebody else's pen has already landed. The check was honest and the answer was stale.

What we built is a noticeboard that physically cannot accept a second name on an occupied line. No supervisor, no looking. Everybody writes at once; the board itself accepts exactly one and refuses the other forty-nine. In software terms the noticeboard is a Postgres table and the physical refusal is an *exclusion constraint* — a rule the database enforces on every write, which cannot be bypassed, forgotten, or raced.

Why that difference matters

Because the rule lives in the database, it holds no matter what reaches it. A bug in our code cannot double-book. A second copy of the application running at the same time cannot double-book. A future developer who forgets the check cannot double-book. There is nothing to remember.

What a student actually sees

- A page listing every facility on campus with how many slots are open today.
- A day grid for a facility, showing each hour as open, taken, theirs, or closed.
- A confirmation with a booking code, or — if they lost the race — a clear reason and three other slots they could take instead.
- A waitlist for a full slot, which is offered to them automatically the moment somebody cancels.

What a facility manager sees

- A console for closing a court for maintenance, which is refused if a student already holds a slot inside that window.
- Analytics: when demand actually peaks, which courts sit empty at prime time, and which bookings are never turned up for.

The demonstration

The application ships with a page that fires the race in front of you. You choose a court, a slot, and how many simultaneous requests to send — up to two hundred — and pick which implementation handles them. Nothing is simulated: both modes write to real Postgres tables through a real HTTP endpoint. The only difference between them is whether the table carries the constraint.

THREE

The concurrency problem, in depth

The brief states the requirement precisely: *“At 6:00 PM, many students request the same facility and time slot at once. The system must confirm exactly one and reject the rest, without corrupting data.”* Four properties follow, and it is worth naming them because they fail independently.

PROPERTY	WHAT IT DEMANDS	HOW IT IS MET HERE
Atomicity	A booking is fully confirmed or not created at all — never half-written.	One transaction; the booking row and its audit row are written by a single statement.
Uniqueness	A facility and time slot has at most one valid booking.	A partial exclusion constraint over <code>(facility_id, during)</code> .
Consistency	Availability and booking records never contradict each other.	Availability is computed from <code>bookings</code> ; there is no second copy to disagree.
Reliability	The result is trustworthy under simultaneous requests.	Verified by a whole-table sweep after every race, at n up to 200.

Anatomy of the race

The failing pattern is four lines long and appears in almost every tutorial:

```
// T1 and T2 arrive within microseconds of each other
const taken = await isSlotTaken(court, slot); // both read: free
if (!taken) await insertBooking(court, slot); // both write
```

The window between the read and the write is not a bug in either line. It is a property of doing two things instead of one. Widening the transaction does not close it, because two transactions can both hold a consistent snapshot in which the slot is free — under `READ COMMITTED`, the default, neither read blocks the other and neither sees the other's uncommitted insert.

Why the usual defences are insufficient

DEFENCE	WHY IT IS NOT ENOUGH
Application-level mutex	Holds only within one process. Vercel runs many concurrent instances; a lock in memory protects nothing.
<code>SERIALIZABLE</code> isolation	Correct, but converts the problem into serialization failures that must be retried, and costs throughput on every transaction rather than only contended ones.
<code>SELECT ... FOR UPDATE</code> on a slot row	Requires a <code>slots</code> table — a second source of truth about occupancy, and one more thing that can disagree with <code>bookings</code> .
<code>UNIQUE (facility_id, starts_at)</code>	Catches identical start times only. It cannot see that 18:30–19:30 overlaps 18:00–19:00, which is the collision that actually happens.

The case a unique key cannot see

A booking from 18:30 to 19:30 does not share a start time with one from 18:00 to 19:00, so a unique index stores both happily. They overlap by thirty minutes. Overlap — not equality — is what matters to a student holding a racket.

FOUR

The mechanism: exclusion constraints over ranges

Postgres can enforce that no two rows in a table satisfy a given pair of operators simultaneously. That is an *exclusion constraint*. With a range type and the overlap operator it expresses precisely the rule this product needs:

```
ALTER TABLE bookings ADD CONSTRAINT bookings_no_overlap
  EXCLUDE USING gist (
    facility_id WITH =,
    during      WITH &&
  ) WHERE (status = 'confirmed');
```

Read aloud: *no two confirmed rows may have the same facility and overlapping time ranges*. Three parts do the work.

during is a tstzrange

A single column holding a half-open interval [*start*, *end*). Half-open matters: a booking ending at 19:00 and one starting at 19:00 do *not* overlap, which is the behaviour a timetable wants. Storing two timestamps instead would push interval logic into application code, where it can be got wrong.

USING gist

B-tree indexes answer equality and ordering. Overlap is neither, so the constraint is backed by a GiST index, which supports the range operators. `btree_gist` is required so that the scalar `facility_id =` clause can live in the same index as the range clause; the migration creates the extension itself.

WHERE (status = 'confirmed')

A *partial* constraint, and this single clause buys two features. Cancelled rows fall out of the index, so cancelling is a status flip rather than a delete: history is preserved and the slot is instantly bookable again with no cleanup job. And because a maintenance closure is stored as a booking row with `kind='block'`, closures pass through the same constraint — a manager cannot close a court out from under a confirmed reservation.

What the loser receives

Postgres raises SQLSTATE `23P01`, `exclusion_violation`. The application catches exactly that code and converts it into a typed product outcome. It is a *result*, not an error: the slot is genuinely gone, so it is never retried.

	UNIQUE (FACILITY, STARTS_AT)	EXCLUDE ... WITH &&
Two bookings at 18:00	rejected	rejected
18:00–19:00 vs 18:30–19:30	both stored	rejected
Two-hour closure spanning four slots	not expressible	one row
Cancelling frees the slot	needs a delete	status flip

FIVE

Data model

Six tables carry the product; two more exist only to support the demonstration. The design principle throughout is that occupancy has exactly one representation.

TABLE	HOLDS
<code>users</code>	The seeded roster — name, role, hostel, roll number, weekly quota, a reliability score used to weight the lottery.
<code>facilities</code>	Twelve courts. Opening and closing time, slot length in minutes, capacity, location, sport, and an <code>is_active</code> flag.
<code>bookings</code>	The only record of occupancy. A facility, an optional user, a <code>tstzrange</code> , a status, a kind (<code>booking</code> or <code>block</code>), an idempotency key, and a human-readable booking code.
<code>waitlist</code>	Queue entries for full slots, with a state machine: <code>queued</code> → <code>offered</code> → <code>claimed</code> , and a claim expiry.
<code>booking_events</code>	Append-only audit. Written in the same statement as the thing it describes, so the trail cannot disagree with reality.
<code>lotteries</code> , <code>lottery_entries</code>	Fair-draw rounds: the published seed, entrant weights, and the drawn order.
<code>rate_limit</code>	Fixed-window counters for the demo endpoints, shared across function instances.
<code>naive_bookings</code> , <code>race_attempts</code>	The control group. Structurally like <code>bookings</code> but with no constraint, so the demo can show what identical traffic does to a schema that trusts application code. It exists in order to fail.

What is deliberately absent

There is no `slots` table. A row per court per hour per day would be a second source of truth about occupancy. The moment it disagreed with `bookings` — a partial write, a missed cleanup, a timezone bug — the brief's consistency requirement would already be lost. Instead the slot grid is generated per request from each facility's own timetable and subtracted against confirmed bookings.

SIX

The write path, statement by statement

A booking request enters at `POST /api/bookings`. The route validates the body with a zod schema, resolves the acting user **from the signed session cookie and never from the request body**, and calls `createBooking`. Inside one transaction, four things happen in order.

1 • One advisory lock, on the user

```
SELECT pg_advisory_xact_lock($userLockKey);
```

Not on the slot — on the person. The quota check that follows is a genuine read-then-write, and one student firing ten parallel requests would otherwise pass ten quota checks and commit all ten. Different students hash to different keys and never contend, so in a race between fifty students this lock is uncontended.

2 • Idempotent replay, resolved first

Before anything is allowed to reject, the transaction looks for a booking already written under this idempotency key by this user. If one exists it is returned as the original. This must sit inside the transaction and behind the advisory lock — that is what makes two simultaneous retries of the same intent serialise, so the second reads the first's committed row instead of racing past it. [Section seven](#) explains why this ordering was a defect for a while.

3 • Quota and self-clash, in one round trip

A single statement returns how many bookings the user holds in the next seven days, their weekly quota, and whether they already hold a booking overlapping this range. A student cannot be on two courts at once — the same shape of rule as the court invariant, scoped to the person. These were two queries once; against a database a continent away every statement inside a transaction is paid for twice, so they were merged.

4 • The decisive statement

```
WITH booked AS (  
  INSERT INTO bookings (facility_id, user_id, during, idempotency_key, ...)  
  VALUES ($1, $2, tstzrange($3, $4, '[')'), $5, ...)  
  RETURNING ...  
) , logged AS (  
  INSERT INTO booking_events (booking_id, facility_id, user_id, type, payload)  
  SELECT b.id, b.facility_id, b.user_id, 'booking.confirmed', ... FROM booked b  
)  
SELECT ... FROM booked;
```

There is **no preceding availability check**. This INSERT either commits and the slot is the user's, or it raises `23P01` and the slot never was. There is no window between deciding and doing, because they

are the same operation. The audit row rides along in the same statement through a CTE — same transaction as before, one round trip instead of two.

Failure handling

SQLSTATE	MEANING	RESPONSE
23P01	Exclusion violation — somebody else holds the slot.	Typed <code>SLOT_TAKEN</code> , three alternatives, waitlist offered. Never retried: it is a settled result.
23505	Unique violation on the idempotency key.	The original booking is returned. Backstop for a retry arriving on another connection before the first commits.
40P01	Deadlock — two transactions each holding a tuple the other must check inside the constraint.	Rolled back cleanly, so it is retried with jittered backoff, six attempts.
40001	Serialization failure.	Retried on the same path.

SEVEN

Idempotency

A transaction cannot distinguish a retry from a second intent. Both are valid requests arriving twice; only the client knows they meant one thing. So the client generates a key per booking intent and sends it. Absent keys are **not** generated server-side — a server-generated key would be fresh on every retry and would defeat the entire mechanism.

On a phone on hostel wifi, a submit that times out is indistinguishable from a submit that failed. The student taps again. Without idempotency they are told they clash with themselves; with it they are shown the booking they already have.

A defect found during verification, and fixed

The replay path originally keyed only off the unique-key violation, which meant it could only fire if the INSERT was reached. It never was: a retry of a successful submit means the caller already holds a confirmed booking over that exact range, so the self-clash pre-check rejected it several statements earlier with **OVERLAPS_OWN**. The branch was unreachable for the one case it existed to serve. Nothing was ever double-booked — the pre-check rejected, so the invariant was never in danger — but the promise was not kept. Resolving the key first, inside the transaction and behind the advisory lock, fixed it. Verified against production with five requests on one key: before, **201** then **409** four times; after, **201** five times returning the same booking code, one row in the table.

EIGHT

Losing well: typed rejections and alternatives

Forty-nine of fifty students lose. An error page tells them nothing. Every rejection here is a typed outcome carrying a reason and a next action.

CODE	CAUSE	CARRIES
SLOT_TAKEN	Lost the race for the slot.	Three alternatives, queue position, waitlist offer.
OVERLAPS_OWN	The student already holds an overlapping booking.	Three alternatives.
QUOTA_EXCEEDED	Weekly quota reached.	The quota and the count.
FACILITY_INACTIVE, OUTSIDE_HOURS, SLOT_MISALIGNED, IN_THE_PAST	Configuration and validation failures, checked before any contended read.	A plain explanation.

Alternatives are computed at rejection time from the same derived availability the grid uses, so they cannot suggest a slot that has just gone. The intent is that losing a race costs a student one tap, not a rethink.

NINE

Cancellation and waitlist promotion

Cancelling flips `status` to `cancelled` and stamps `cancelled_at`. Because the exclusion constraint is partial over confirmed rows, the row leaves the index the instant it commits and the slot is bookable again — no delete, no cleanup job, and the history is retained.

Ownership is enforced in the statement itself: the `UPDATE` is scoped by `user_id`, so a forged booking id matches no row and returns `404` rather than cancelling somebody else's court.

Promotion happens in the same transaction

Releasing the slot and offering it to the next student in the queue are one atomic operation. A background job would leave a window in which the slot is free but unowned — precisely the window in which a race would take it, and precisely the bug this whole system exists to prevent. The offer carries an expiry; claiming it re-checks that expiry inside a `SELECT ... FOR UPDATE` rather than trusting what the page rendered seconds ago.

The claim is then a normal constrained INSERT. An offer is a promise that the slot was free when the offer was made, not that it still is — a manager may have closed the court for maintenance in between. If the claim loses, the student is told plainly rather than handed a booking that does not exist.

TEN

Maintenance closures

A closure is a `bookings` row with `kind='block'` and no user. One invariant, two features: because it travels the same constraint, a manager attempting to close a window that a student already holds is refused with the same `23P01`, and the console names who is in the way.

The constraint does not care who is in the way, but the manager does — so the response distinguishes an existing closure (a duplicate action) from a student booking (a conflict with a person), because those need different next steps.

Reopening is a status flip guarded on `kind='block'`, so the endpoint cannot be pointed at a student's booking. It writes a `block.lifted` event to the audit log.

ELEVEN

The fair draw

Winning a race is not the same as deserving the court. First-come-first-served rewards whoever has the best wifi and the fastest thumb. For genuinely contended peak slots the product offers an alternative: a seeded, weighted, published lottery.

- **The seed is published before the draw**, alongside the rules and the entrant list, so anyone can recompute the winner afterwards and check it.
- **Weights are stated**: reliability score (a student who turns up is weighted higher than one who no-shows) and how recently the entrant last won a peak slot.
- **The draw is deterministic** given the seed and the entrant set, which is what makes it auditable rather than merely random.

The winner's booking is then written through the same constrained path as any other booking. The lottery decides *who may attempt* the write; it does not get to bypass the invariant.

TWELVE

Availability is derived, never stored

For a requested facility and date, the slot grid is generated from that facility's own opening time, closing time and slot length using `generate_series`, then each candidate slot is tested against confirmed bookings with `NOT EXISTS (... during && ...)`. The same query labels a slot as the caller's own where the booking's user matches the session.

Everything is computed in `Asia/Kolkata`. Slot boundaries are derived from the facility row rather than assumed to be hourly, so a ninety-minute cricket booking and a thirty-minute squash booking coexist without special cases.

The consequence

Availability cannot drift from bookings, because it is not a separate fact. Cancel a booking and the next request already reflects it. There is no materialised view to refresh, no cache to invalidate, and no nightly job whose failure quietly corrupts the front page.

THIRTEEN

Real time, and why the payload is a hint

Open day grids update live over Server-Sent Events at `GET /api/stream`. The event payload is deliberately minimal — “something changed on this facility” — and clients respond by re-fetching the authoritative day view rather than patching local state from the message.

That way a dropped, duplicated or out-of-order event can never leave a screen disagreeing with the database; the worst it can do is cost one extra fetch. A fifteen-second poll runs alongside as a fallback for clients whose SSE connection cannot be established, and a keep-alive comment frame stops proxies dropping the idle connection.

Publication happens after the writer has committed. It is a notification, never part of the transaction — a failed publish must never be able to roll back a confirmed booking.

FOURTEEN

The interface

Set as a printed institutional prospectus rather than a dashboard. There is deliberately **no brand hue**: chrome is charcoal, paper is a warm off-white, and the only saturated colours on screen are the ones that mean something.

TOKEN	VALUE	ROLE
ink	#22201e	Chrome, headings, primary action
rust	#b4512a	The one call to action, and the live state
brass	#c29a4b	Hairline emphasis; never a fill
paper	#f7f4ee	The ground
forest / brick	#2e6b3e / #8c2f22	Status only — independent of brand

Every grey is warm, mixed toward the paper rather than toward blue. Cool grey on cool grey is the strongest visual tell of a generated interface.

State is carried by fill, not by hue

Open is paper, taken is solid charcoal, the reader's own is rust, closed is hatched — so the grid stays readable in greyscale, to a colourblind reader, and through a badly calibrated projector. Availability is stated three ways on every card: a coloured dot, a word, and a count. Hue is never the only channel.

Motion

Motion is a response, never decoration. Cards rise into view once as the reader reaches them, driven by a single document-wide IntersectionObserver so that server-rendered markup can opt in with a bare attribute. The hidden start state is gated behind an attribute the script sets, so with JavaScript off nothing is hidden — an animation is never the reason a page is blank. Everything is dropped under `prefers-reduced-motion`, except the cues that carry meaning, which remain present but unanimated.

FIFTEEN

Deployment and topology

LAYER	CHOICE
Framework	Next.js 15 App Router, React 19, TypeScript in strict mode.
Runtime	Node.js serverless functions on Vercel, pinned to region <code>sin1</code> (Singapore).
Database	Neon Postgres 17 in <code>ap-southeast-1</code> , with <code>btree_gist</code> .
Driver	<code>postgres</code> (postgres.js). Tagged templates only, so every value is a bind parameter.
Styling	Tailwind v4 with a token layer; no component library.

Region is a correctness-adjacent decision

Functions and database sit in the same region on purpose. Every statement inside a transaction costs a round trip out and back; with the database a continent away, a transaction containing four statements pays four times the latency. Co-locating them is what makes a 200-way burst finish inside the function's time budget.

The connection pool

The pool is created lazily on first use, never at module import. Next statically generates the 404 page, which pulls in the root layout — a pool created at import time would fail any build without a `DATABASE_URL`, such as CI or a judge cloning the repository to read it. Prepared statements are disabled when the connection string indicates a pooler, which cannot guarantee the same backend across statements.

The shell renders without a database

The layout's data load is wrapped so that a throwing query yields an empty shell rather than a failed build. Pages that genuinely need data fail on their own, where the error is actionable.

SIXTEEN

Performance engineering

The per-slot lock that was removed

An earlier version took a second advisory lock on the slot, in sorted key order alongside the user lock. On a co-located database it was faster: contenders queued on a cheap lock instead of piling into the constraint's wait-for-uncommitted-row path.

Over a network it was a disaster, because it serialised every contender and each transaction then cost a full round-trip budget. Measured against Neon from a Vercel function:

CONTENDERS	2	5	10	20
With a per-slot lock	6.3 s	10.6 s	19.2 s	36.4 s

Dead linear at about 1.8 s per contender; fifty would have exceeded the function timeout.

Correctness was never in question — one booking, every run — throughput was. Removing it lets all contenders attempt the insert at once: one wins, the rest block on its uncommitted row and are released together the moment it commits. Total time becomes one transaction plus a round trip, whatever n is.

Removing it also removed the deadlock class it was guarding against. A deadlock needs two locks taken in varying order; with a single lock per transaction, keyed per user, an advisory-lock cycle cannot be constructed.

The residual deadlock, and the retry that absorbs it

Partially-overlapping ranges can still deadlock inside the exclusion constraint itself, when two transactions each hold a tuple the other must check. That rolls back cleanly, so it is retried. Three attempts backing off 15/30/60 ms proved measurably insufficient — the load suite escaped it in CI while eighteen of nineteen tests passed — so the wrapper now allows six attempts with wider jitter, backing off to about a second in total. Surfacing a deadlock to a student who did nothing wrong is far more expensive than waiting.

Measured throughput

MODE	REQUESTS	SERVER TIME	CONFIRMED	ROWS	OVERLAPPING PAIRS
naive	50	37 ms	50	50	1 225
safe	10	47 ms	1	1	0
safe	50	116 ms	1	1	0
safe	100	251 ms	1	1	0
safe	200	546 ms	1	1	0

Median of three warm runs. Twenty times the load for roughly ten times the time — sub-linear, because nothing in the write path serialises on n .

Cold start is real, and worth knowing before a demo

The first race fired after an idle period runs roughly three times slower than the table above while the function and the connection pool warm up. Measured cold: 149 ms at $n=10$ against 47 ms warm. Fire one race before presenting.

SEVENTEEN

Verification strategy

Every claim in this document is reproducible from the repository. Four independent layers check different things.

1 · Unit and concurrency suites — `npm run test`

Nineteen tests under vitest, including a load test that fires concurrent bookings at one slot. CI runs them against a real Postgres 17 service container rather than a mock, because a mock cannot exhibit the behaviour under examination.

2 · The whole-table invariant sweep — `npm run invariant`

Every confirmed row checked against every other row on the same facility. Not a sample, and not limited to the slot just tested. Exits non-zero if it finds a single overlapping pair, so it can gate CI or be run against production as a smoke check. The same sweep runs server-side after every race and its result is returned in the response.

3 · Browser journey — `npm run e2e`

Eleven checks driving a real browser through the student journey: browse, open a slot grid, open the booking sheet, confirm, see the code in My Bookings, see the slot marked as theirs, cancel, and confirm the slot returns to the grid as open. It asserts zero uncaught client errors throughout, and can be pointed at any deployment with `E2E_BASE`. **Result against production: 11/11.**

4 · Continuous integration

On every push: typecheck, apply the schema to a live Postgres, seed, run the suites, run the invariant sweep, and produce a production build.

A test bug found and fixed during this work

The browser suite located a bookable slot by searching for a button whose text contained “Open”. A redesign gave the header a mobile menu toggle labelled “Open menu”, which sorts earlier in the document, so the suite clicked the hamburger and timed out waiting for a booking dialog. The product was fine; the selector was not. Slots now expose `data-slot-state` and the test addresses that instead. The post-cancellation recount had the same flaw and was passing only because both sides of its comparison were wrong by the same one.

EIGHTEEN

Security posture and threat model

What is enforced

CONTROL	IMPLEMENTATION
Ownership	Cancellation scopes to <code>user_id</code> inside the <code>UPDATE</code> ; waitlist claims scope inside a <code>SELECT ... FOR UPDATE</code> . A forged id matches no row.
Actor identity	Derived from the signed cookie on the server on every write path, never from the request body.
Input validation	A zod schema on every route body before anything reaches SQL.
Injection	Every query is a tagged template, so every value is a bind parameter. There is no <code>sql.unsafe</code> in application code.
Cookie integrity	HMAC-SHA256 signed, <code>httpOnly</code> , <code>sameSite=lax</code> , <code>secure</code> in production, compared with <code>timingSafeEqual</code> .
Response headers	<code>X-Frame-Options: DENY</code> , <code>nosniff</code> , a referrer policy and a permissions policy on every response.
Dependencies	<code>npm audit</code> : 0 vulnerabilities.

What is deliberately not enforced

There are no passwords. Identity is a signed cookie over a seeded roster with a “switch reader” control, because racing two students against one slot has to be doable in front of a judge without two browser profiles. The cost, stated rather than left to be discovered:

SURFACE	EXPOSURE
<code>POST /api/session</code>	Issues any identity with no credential, and roster ids appear in the page HTML — so anyone can become a manager. The role check on <code>/api/ops</code> is therefore a demo guard, not a security boundary. Verified: a malformed body returns <code>400</code> as a manager and <code>403</code> anonymously, which proves the gate is passed.
<code>/api/race</code> , <code>/api/race/reset</code> , <code>/api/lottery</code>	Unauthenticated. Rate limited rather than gated, because the race demo has to stay one click away.

The rate limiter, and a negative result worth recording

The first implementation kept counters in module memory. Measured against the deployment, it did nothing at all: sixty-six consecutive requests under a single key were every one of them allowed,

because the platform did not reuse the function instance and each request began with an empty map. A counter that never counts is worse than no counter, because it reads like protection.

It now counts in Postgres — the only state every instance already shares — with a single upsert that increments and returns the new count, so two simultaneous callers cannot both read “under the limit”. It **fails open**: a missing table or failed query allows the request, because a demo that refuses to run when its rate limiter breaks has turned a nice-to-have into an outage. It is not an authentication boundary; `x-forwarded-for` is client-controlled and the key can be rotated freely.

Swapping in real authentication is a contained change: replacing `currentUser()` is the whole job, because no call site reads the cookie directly.

NINETEEN

Limitations, and what we would do next

LIMITATION	NEXT STEP
No real authentication.	Institute SSO behind <code>currentUser()</code> ; every write path already takes its actor from the server.
Rate limiting keyed on a client-controlled header, and inactive until the table is created.	Key on authenticated identity; run the migration as part of deploy.
No Content-Security-Policy.	Next's inline bootstrap needs per-request nonces — a change to the rendering path, not a config line, and one worth testing rather than bolting on.
Waitlist offers are visible only in the application.	Push or email notification, so an offer that expires in ten minutes reaches somebody who is not looking at the page.
Single region.	Read replicas for availability queries; the write path must stay next to the primary.
Analytics are computed per request.	Materialise the heavier aggregates once demand justifies it — carefully, since a second source of truth is exactly what this design avoids elsewhere.

What we would not change

The constraint. Every feature in this document — closures, cancellation, waitlist promotion, the lottery — is expressed through it rather than around it. That is the reason the system has one invariant to reason about instead of five, and the reason a future contributor cannot accidentally introduce a double booking.

APPENDIX A

API reference

ENDPOINT	BEHAVIOUR
GET /api/availability	Derived day grid for <code>?facility=&date=</code> . 400 without both, 404 for an unknown facility. Polling fallback for clients without SSE.
POST /api/bookings	Create a booking. Body: facility, start, end, idempotency key, optional party size and note. 201 confirmed, or a typed rejection.
POST /api/bookings/cancel	Cancel by id, scoped to the session user. 404 if not theirs.
POST /api/waitlist	Join the queue for a taken slot; returns queue position.
POST /api/waitlist/claim	Claim an offer. Re-checks expiry under lock, then a normal constrained insert.
POST /api/ops	Manager only: <code>block</code> , <code>unblock</code> , <code>toggle</code> . Closures pass the booking constraint.
POST /api/lottery	<code>simulate</code> seeds a round; <code>draw</code> runs the seeded weighted draw. GET <code>?id=</code> returns the round.
POST /api/race	Fire n concurrent bookings in <code>safe</code> or <code>naive</code> mode; returns per-request outcomes and a whole-table sweep.
POST /api/race/reset	Clears demo rows only — keys matching <code>race-%</code> plus the two demo tables.
GET /api/stream	Server-sent events. Payload is a hint; clients re-fetch.
POST /api/session	Switch reader. No credential — see section eighteen.

APPENDIX B

Measured data

Concurrency, safe mode — three warm runs each

N	RUN 1	RUN 2	RUN 3	MEDIAN	CONFIRMED	OVERLAPS
10	43	47	47	47 ms	1	0
50	116	191	95	116 ms	1	0
100	221	313	251	251 ms	1	0
200	364	546	563	546 ms	1	0

Eighteen safe runs were executed in total during verification. Every one ended with exactly one confirmed booking and zero overlapping pairs table-wide.

Reads while a 200-way burst is in flight

PAGE	IDLE	UNDER LOAD	STATUS
Home	249–378 ms	625 ms	200
Facility grid	—	378 ms	200
Availability API	—	291 ms	200
My bookings	—	420 ms	200

The race itself still returned 1 confirmed, 199 rejected, 0 overlaps in 928 ms. A judge clicking around during the demo sees no errors and no hang.

Backend probe against production

Sixteen checks, all as expected: availability (200 / 400 / 404), malformed bodies rejected 400, idempotent replay returning one booking across five requests, typed rejection with three alternatives, forged cancellation 404, SSE ready frame, and the privilege-escalation finding confirmed rather than asserted.

APPENDIX C

Repository map and glossary

migrations/0001_init.sql	the schema — and the constraint that is the product
src/db/client.ts	pool, SQLSTATE constants, deadlock retry
src/lib/booking.ts	the write path: create, cancel, promote, suggest
src/lib/race.ts	the race harness and the whole-table invariant sweep
src/lib/lottery.ts	seeded weighted draw, deterministic and auditable
src/lib/availability.ts	derived availability — no slots table
src/lib/ratelimit.ts	fixed-window counters, in Postgres, fail-open
src/lib/session.ts	signed-cookie identity; the single swap point for SS0
src/app/	Next.js App Router: pages and route handlers
src/components/	UI, court diagrams, sport glyphs
tests/	vitest — concurrency and lottery suites
scripts/	migrate · seed · invariant · e2e · media · deck · report
docs/ARCHITECTURE.md	data model, trade-offs, what we would do next
docs/DEPLOY.md	Neon + Vercel, and the region that matters

Glossary

TERM	MEANING
Exclusion constraint	A Postgres constraint forbidding two rows from simultaneously satisfying a set of operator comparisons.
tstzrange	A timestamp-with-timezone range type. Used half-open, <code>[start, end)</code> .
GiST	Generalised Search Tree — the index type able to answer overlap queries, unlike a B-tree.
&&	The range overlap operator.
23P01	SQLSTATE for <code>exclusion_violation</code> — the loser of a race.
40P01	SQLSTATE for a detected deadlock. Rolled back cleanly, therefore retryable.
Advisory lock	An application-defined lock held by Postgres. Used here per user, never per slot.
Idempotency key	A client-generated token identifying one booking intent, so a retry returns the original result.
Partial constraint	A constraint with a <code>WHERE</code> clause, so only qualifying rows are indexed — here, only confirmed bookings.

Reproducing every figure in this report

```
npm run test · npm run invariant · npm run e2e · E2E_BASE=<url> npm run e2e
```